```
Enter choice: 11
Queue (Front to Rear):
ID: 105 | Name: Karan | Salary: 27000.00

Enter choice: 10
Dequeued employee: ID: 105 | Name: Karan
| Salary: 27000.00

Enter choice: 0
Program terminated.
```

```
Enter choice: 3

Employees (End to Beginning):
ID: 103 | Name: Amit | Salary: 28000.00
ID: 102 | Name: Priya | Salary: 30000.00
ID: 101 | Name: Rahul | Salary: 25000.00

Enter choice: 6

Enter Employee ID: 104
Enter Name: Sneha
Enter Salary: 32000
Employee pushed onto stack.

Enter choice: 8
Stack (Top to Bottom):
ID: 104 | Name: Sneha | Salary: 32000.00

Enter choice: 9

Enter Employee ID: 105
Enter Name: Karan
Enter Salary: 27000
Employee enqueued successfully.
```

```
Enter choice: 1

Enter Employee ID: 102
Enter Name: Priya
Enter Salary: 30000
Employee added to directory.

Enter choice: 1

Enter Employee ID: 103
Enter Name: Amit
Enter Salary: 28000
Employee added to directory.

Enter choice: 2

Employees (Beginning to End):
ID: 101 | Name: Rahul | Salary: 25000.00
ID: 102 | Name: Priya | Salary: 30000.00
ID: 103 | Name: Amit | Salary: 28000.00
```

```
===== EMPLOYEE DIRECTORY =====
1. Add employee to directory
2. Display directory forward
3. Display directory backward
4. Add to singly linked list
5. Display singly linked list
6. Push onto stack
7. Pop from stack
8. Display stack
9. Enqueue into queue
10. Dequeue from queue
11. Display queue
0. Exit
Enter choice: 1

Enter Employee ID: 101
Enter Name: Rahul
Enter Salary: 25000
Employee added to directory.
```

1. Problem Statement

Develop an Employee Directory Management System using data structures in C. The system should store employee records such as Employee ID, Name, and Salary using a **doubly linked list**. It should also demonstrate a **singly linked list, stack, and queue** with enqueue, dequeue, and display operations.

The program should provide the following operations:

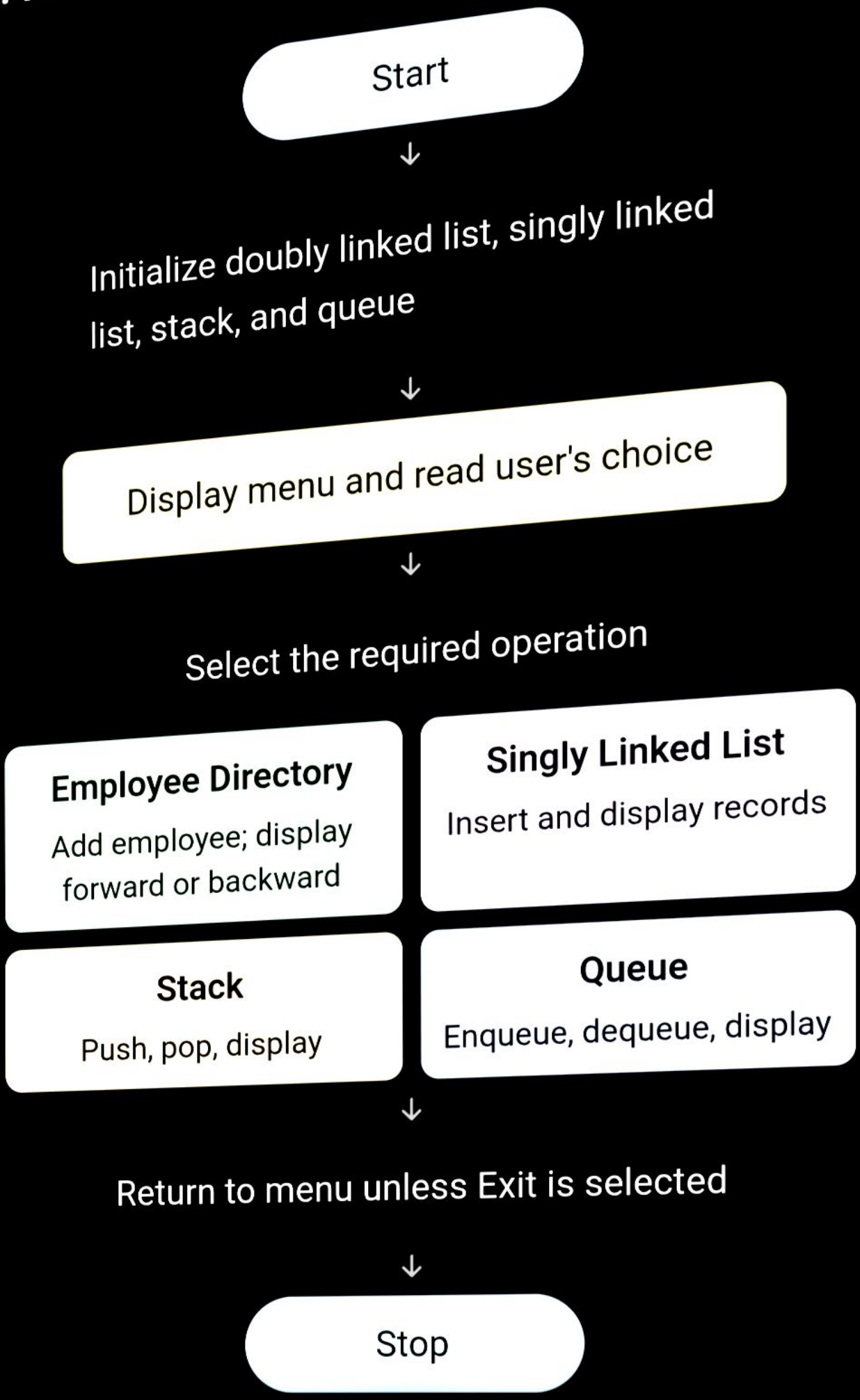
- Add and store employee records.
- Display employee records from beginning to end (forward traversal).
- Display employee records from end to beginning (backward traversal).
- Demonstrate stack operations such as push, pop, and display.
- Demonstrate queue operations such as enqueue, dequeue, and display.
- Use linked lists to manage records dynamically.

2. Algorithm

1. Start the program.
2. Define an employee node containing Employee ID, Name, Salary, and pointers to the previous and next nodes.
3. Initialize the head and tail pointers to `NULL`.
4. Display a menu with the available operations.
5. Accept the user's choice.
6. Perform the selected operation:
    - **Add employee:** Create a new node and insert it at the end of the doubly linked list.
    - **Forward display:** Traverse from head to tail.
    - **Backward display:** Traverse from tail to head.
    - **Stack push:** Insert an employee record at the top of the stack.
    - **Stack pop:** Remove the top employee record.
    - **Queue enqueue:** Insert an employee record at the rear of the queue.
    - **Queue dequeue:** Remove an employee record from the front of the queue.
    - **Display queue:** Traverse the queue from front to rear.
7. Repeat until the user selects Exit.
8. Free the dynamically allocated memory.
9. Stop.



3. Flowchart


Start
Initialize doubly linked list, singly linked list, stack, and queue
Display menu and read user's choice
Select the required operation
Employee Directory
Add employee; display forward or backward
Singly Linked List
Insert and display records
Stack
Push, pop, display
Queue
Enqueue, dequeue, display
Return to menu unless Exit is selected
Stop